

Explanation - Utkarsh Aditya

Reliable Data Transfer over UDP (CS-30003 CA1, P3)

Member: Utkarsh Aditya (24051069) | Part: Go-Back-N, cumulative ACKs, sliding window (+ main.py that joins everything)

Your files: utkarsh_aditya/go_back_n.py, main_app/main.py, main_app/config.py

The 1-minute story

- Stop-and-Wait wastes time waiting. **Go-Back-N (GBN)** fixes this with a **sliding window**: the sender may have up to N packets 'in the air' without waiting for ACKs (like sending 8 letters and then waiting for confirmations).
- **Cumulative ACK**: the receiver sends ACK n meaning 'I have received EVERYTHING below n, please send n next'. One ACK confirms many packets.
- The receiver is simple: it only accepts packets **in order**. A packet that arrives out of order is thrown away.
- The sender has **one timer** for the oldest unacknowledged packet. If it expires, the sender 'goes back' and resends **the whole window** starting from that oldest packet. That is where the name comes from.
- Weakness: one lost packet makes us resend many good packets too. So GBN wastes bandwidth when loss is high. That is exactly what our experiment shows (SR beats GBN).

Window picture (window = 4)

Packets: 0 1 2 3 4 5 6 7 ...

[0 1 2 3] are sent. base = 0 (oldest not acked), next_seq = 4 (next new one).

ACK 2 arrives (cumulative: 0 and 1 done) --> base = 2 --> window slides: now [2 3 4 5], so 4 and 5 can be sent.

If packet 2 is lost: receiver discards 3,4,5 (out of order) and keeps saying 'ACK 2'. Timer expires --> resend 2,3,4,5.

Variables to remember

- **base** = oldest packet not yet acknowledged (left edge of the window).
- **next_seq** = next new packet to send (right edge). Packets in flight = next_seq - base <= window.
- **expected** (receiver) = the only sequence number the receiver will accept next.
- **timer_start** = time when the timer for the oldest packet began.

main.py in one paragraph

main.py is the 'control room'. It reads the file, cuts it into 1024-byte chunks, creates two UDP sockets (sender and receiver), wraps both in Shlok's channel emulator, starts the receiver in a background thread, runs the sender, measures time, saves the received file and compares SHA-256 hashes. The function `run_transfer()` does all this so both the command line and the experiment script can use it.

File: utkarsh_aditya/go_back_n.py

Code	What it means (plain English)
<code>import time</code>	For timers.
<code>from packet import make_packet, receive_packet, DATA, ACK, FIN, GIVE_UP_SECONDS</code>	Reuse Arijeet's packet tools.
<code>def send(chan, dest, chunks, window, timeout, adaptive=True):</code>	Sender. window = how many packets may be in flight. timeout = timer length. (adaptive is only used by Selective Repeat.)
<code> items = [(DATA, c) for c in chunks] + [(FIN, b'')]</code>	File pieces as DATA packets plus a final FIN packet.
<code> total = len(items)</code>	Total number of packets to deliver.
<code> packets = [make_packet(t, seq, p) for seq, (t, p) in enumerate(items)]</code>	Build ALL packets' bytes in advance and keep them in a list, so that resending is just re-sending packets[seq].
<code> base = 0 # oldest packet not yet acknowledged</code>	Window's left edge: oldest unacknowledged packet.
<code> next_seq = 0 # next new packet to send</code>	Next new packet to send.
<code> timer_start = time.time() # one timer, for the oldest unacked packet</code>	The single timer. We remember when it started.
<code> last_progress = time.time()</code>	Last time an ACK moved the window (used for the give-up safety check).
<code> packets_sent = 0</code> <code> retransmissions = 0</code>	Counters for the experiment results.
<code> while base < total:</code>	Keep going until every packet is acknowledged (base has moved past the last packet).
<code> while next_seq < total and next_seq < base + window:</code>	STEP 1, fill the window: while there are still packets left AND fewer than `window` packets are in flight, send more.
<code> chan.sendto(packets[next_seq], dest)</code> <code> packets_sent += 1</code>	Send the packet through the emulator and count it.
<code> if base == next_seq: # window was empty -> start the timer</code> <code> timer_start = time.time()</code>	If nothing was in flight before this packet, this packet is now the oldest, so start the timer now.
<code> next_seq += 1</code>	Window's right edge moves by one.
<code> remaining = timeout - (time.time() - timer_start)</code>	STEP 2, how much of the timer is left? (timeout minus time already elapsed.)
<code> reply = receive_packet(chan, remaining) if remaining > 0 else None</code>	Wait for an ACK, but only for the remaining timer time. If the timer already ran out, don't wait: treat it as no reply.
<code> if reply is not None:</code>	We got some good packet.
<code> reply_type, ack_number, _ = reply</code>	Unpack it. For ACKs, the sequence field holds the cumulative ACK number.

Code	What it means (plain English)
<code>if reply_type == ACK and base < ack_number <= next_seq:</code>	Valid new cumulative ACK? It must be an ACK and larger than base (progress) but not bigger than what we have sent (sanity). Old duplicate ACKs (<code>ack_number <= base</code>) are ignored.
<code>base = ack_number # slide the window forward</code>	SLIDE THE WINDOW: everything below <code>ack_number</code> is confirmed, so the oldest unacked packet is now <code>ack_number</code> .
<code>oldest packet timer_start = time.time() # restart timer for the new</code>	Restart the timer for the new oldest packet.
<code>last_progress = time.time()</code>	Note that we made progress.
<code>elif time.time() - timer_start >= timeout:</code>	No useful reply AND the timer has expired -> TIMEOUT.
<code>for seq in range(base, next_seq):</code>	STEP 3, go back: loop over every packet in the window (oldest to newest).
<code>chan.sendto(packets[seq], dest) packets_sent += 1 retransmissions += 1</code>	Resend each one and count it as a retransmission. (This is why GBN wastes packets: good ones are resent too.)
<code>timer_start = time.time()</code>	Restart the timer after the retransmission burst.
<code>if time.time() - last_progress > GIVE_UP_SECONDS: raise TimeoutError("go-back-n: receiver not responding")</code>	Safety: no progress for 20 s means the receiver is gone; stop with an error.
<code>return {"packets_sent": packets_sent, "retransmissions": retransmissions}</code>	Send the counters back to main.py.
<code>def receive(chan, dest, window, linger):</code>	Receiver side of GBN.
<code>expected = 0 # next in-order sequence number we want</code>	The only sequence number we will accept next.
<code>chunks = [] finished = False</code>	chunks = file pieces in order. finished = FIN seen?
<code>while True:</code>	Receive loop.
<code>wait = linger if finished else GIVE_UP_SECONDS</code>	After FIN wait only a short 'linger' time (to re-ACK if our last ACK was lost).
<code>packet = receive_packet(chan, wait)</code>	Get the next good packet.
<code>if packet is None: if finished: return chunks raise TimeoutError("go-back-n: sender went silent")</code>	Silence after FIN = done, return the data. Silence before FIN = error.
<code>packet_type, seq, payload = packet</code>	Unpack.
<code>if packet_type == ACK: continue</code>	Ignore ACK packets (not meant for the receiver).
<code>if seq == expected: # in order -> accept</code>	Is it the exact in-order packet we want?

Code	What it means (plain English)
<pre>if packet_type == FIN: finished = True else: chunks.append(payload)</pre>	FIN -> remember we're done. DATA -> store the piece.
<pre>expected += 1</pre>	Now we want the next number.
<pre>chan.sendto(make_packet(ACK, expected), dest)</pre>	ALWAYS reply with a cumulative ACK = 'next packet I expect'. If a packet was out of order (skipped by the if above) this repeats the previous ACK, which tells the sender 'I'm still missing expected'. Out-of-order packets are simply discarded - that's Go-Back-N.

File: main_app/config.py

Code	What it means (plain English)
<code>HOST = "127.0.0.1" # everything runs on this computer</code>	127.0.0.1 = 'this same computer' (localhost). Everything runs locally.
<code>PAYLOAD_SIZE = 1024 # file bytes carried in each packet</code>	Each packet carries up to 1024 bytes of file data.
<code>DEFAULT_TIMEOUT = 0.1 # seconds to wait for an ACK before resending</code>	Wait 0.1 s for an ACK before resending.
<code>DEFAULT_WINDOW = 8 # window size for GBN and SR</code>	Default window size for GBN and SR.
<code>LINGER = 1.0 # receiver waits this long after FIN (in case last ACK was lost)</code>	Receiver stays alive 1 s after FIN.
<code>DEFAULT_DELAY = 0.01 # one-way network delay (10 ms)</code>	Emulated one-way network delay: 10 ms.
<code>DEFAULT_JITTER = 0.0 # extra random delay (0 = off, so reordering only happens when we ask for it)</code>	Extra random delay (jitter). 0 = off, so packets never get reordered by accident; reordering only happens when we request it with --reorder. Set e.g. 0.005 to add up to 5 ms of random jitter.

File: main_app/main.py

Code	What it means (plain English)
<pre>import argparse import os import socket import sys import threading import time</pre>	argparse = read command-line options. os/sys = file paths and import paths. socket = UDP. threading = run receiver at the same time as sender. time = measure duration.
<pre>ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))</pre>	Find the project's top folder: this file's folder (main_app), then one level up.
<pre>for folder in ["main_app", "arijeet_kundu", "utkarsh_aditya", "vedant_bhatnagar", "shlok_jaiswal"]:</pre>	Go through all five folders. Since every member's code is in a separate folder, Python needs to be told where to look.
<pre> sys.path.insert(0, os.path.join(ROOT, folder))</pre>	Add each folder to Python's import search path. After this, 'import stop_and_wait' works even though it sits in another folder.
<pre>import config import stop_and_wait import go_back_n import selective_repeat</pre>	Import the settings and the three protocol modules (one from each member).
<pre>from channel_emulator import ChannelSocket from file_hash import files_match</pre>	Shlok's bad-network emulator and SHA-256 comparison.
<pre>PROTOCOLS = { "stop-and-wait": stop_and_wait, "gbn": go_back_n, "sr": selective_repeat, }</pre>	A dictionary mapping the name typed by the user to the module. Because all three modules have functions called send() and receive() with the same parameters, we can swap protocols without changing any other code (the 'pluggable' idea in our proposal).
<pre>def run_transfer(protocol, input_path, output_path, window=config.DEFAULT_WINDOW, loss=0.0, duplicate=0.0, reorder=0.0, corrupt=0.0, delay=config.DEFAULT_DELAY, jitter=config.DEFAULT_JITTER, seed=1, timeout=config.DEFAULT_TIMEOUT, adaptive=True):</pre>	The main function: one complete file transfer. The parameters are the protocol, files, window size, and the channel's bad behaviour (loss, duplicate, reorder, corrupt probabilities, delay, jitter), a random seed and the timeout.
<pre> module = PROTOCOLS[protocol]</pre>	Pick the chosen protocol's module.
<pre> with open(input_path, "rb") as f: data = f.read()</pre>	Read the whole input file as bytes.
<pre> size = config.PAYLOAD_SIZE</pre>	1024.
<pre> chunks = [data[i:i + size] for i in range(0, len(data), size)]</pre>	Cut the data into pieces of 1024 bytes: data[0:1024], data[1024:2048], ... (last one may be shorter).
<pre> sender_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) receiver_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)</pre>	Create two UDP sockets. AF_INET = IPv4, SOCK_DGRAM = UDP.
<pre> sender_sock.bind((config.HOST, 0)) receiver_sock.bind((config.HOST, 0))</pre>	Attach each socket to localhost. Port 0 means 'operating system, please choose any free port', so we never get 'port already in use' errors.

Code	What it means (plain English)
<pre>sender_addr = sender_sock.getsockname() receiver_addr = receiver_sock.getsockname()</pre>	Ask the OS which (IP, port) each got. The sender needs the receiver's address and vice-versa (for ACKs).
<pre>settings = dict(loss=loss, duplicate=duplicate, reorder=reorder, corrupt=corrupt, delay=delay, jitter=jitter)</pre>	Bundle the channel settings in one dictionary so we can reuse them for both directions.
<pre>sender_chan = ChannelSocket(sender_sock, seed=seed, **settings) receiver_chan = ChannelSocket(receiver_sock, seed=seed + 1000, **settings)</pre>	Wrap each socket in the emulator. Now everything sent (DATA one way, ACKs the other way) passes through the bad network. Different seeds for the two directions, but the same seed always gives the same behaviour. ** unpacks the dictionary into keyword arguments.
<pre>outcome = {}</pre>	Empty dictionary where the receiver thread will leave its result.
<pre>def receiver_job(): try: outcome["chunks"] = module.receive(receiver_chan, sender_addr, window, config.LINGER) except Exception as error: outcome["error"] = error</pre>	A small function run by the thread: call the protocol's receive(), store the file pieces it returns. If something goes wrong, save the error instead of crashing silently.
<pre>thread = threading.Thread(target=receiver_job) thread.start()</pre>	Start the receiver in the background so sender and receiver run at the same time (like two people on two computers).
<pre>start = time.time()</pre>	Note the start time.
<pre>stats = module.send(sender_chan, receiver_addr, chunks, window, timeout, adaptive)</pre>	Run the SENDER (blocks until all packets are acknowledged). Returns packets_sent and retransmissions.
<pre>elapsed = time.time() - start</pre>	Total transfer time in seconds.
<pre>thread.join()</pre>	Wait for the receiver thread to finish.
<pre>sender_chan.close() receiver_chan.close()</pre>	Close both sockets.
<pre>if "error" in outcome: raise outcome["error"]</pre>	If the receiver failed, show the error now.
<pre>with open(output_path, "wb") as f: f.write(b"".join(outcome["chunks"]))</pre>	Join all received pieces into one byte string and write the output file.
<pre>stats.update({</pre>	Add extra information to the stats dictionary:
<pre> "protocol": protocol, "bytes": len(data), "time": elapsed,</pre>	Protocol name, file size in bytes and the time taken.
<pre> "goodput": len(data) / elapsed if elapsed > 0 else 0, # useful bytes per second</pre>	GOODPUT = useful file bytes delivered per second (file size / time). Retransmitted bytes do not count as useful, unlike 'throughput'.
<pre> "intact": files_match(input_path, output_path),</pre>	SHA-256 of the original file vs the received file: True if they are identical.
<pre> "dropped": sender_chan.dropped + receiver_chan.dropped, })</pre>	How many packets the emulator dropped in both directions.

Code	What it means (plain English)
<code>return stats</code>	Give the results back to whoever called <code>run_transfer</code> .
<code>def main():</code>	Command-line entry point.
<code>parser = argparse.ArgumentParser(description="Reliable file transfer over UDP")</code>	Create a parser that reads options from the command line.
<code>parser.add_argument("--protocol", choices=list(PROTOCOLS), required=True)</code> <code>parser.add_argument("--input", required=True)</code> <code>parser.add_argument("--output", default="received_output.txt")</code>	Options: protocol (must be one of our three), the input file, and the output file name.
<code>parser.add_argument("--window", type=int, default=config.DEFAULT_WINDOW)</code> <code>parser.add_argument("--loss", type=float, default=0.0)</code> <code>parser.add_argument("--duplicate", type=float, default=0.0)</code> <code>parser.add_argument("--reorder", type=float, default=0.0)</code> <code>parser.add_argument("--corrupt", type=float, default=0.0)</code> <code>parser.add_argument("--seed", type=int, default=1)</code> <code>parser.add_argument("--timeout", type=float, default=config.DEFAULT_TIMEOUT)</code>	Window size and the channel's bad behaviour, e.g. <code>--loss 0.1</code> means 10% of packets are lost. Seed makes runs repeatable.
<code>args = parser.parse_args()</code>	Read what the user typed.
<code>r = run_transfer(args.protocol, args.input, args.output, args.window, args.loss, args.duplicate, args.reorder, args.corrupt, seed=args.seed, timeout=args.timeout)</code>	Do the transfer with those options.
<code>print("Protocol :", r["protocol"])</code> <code>print("File size :", r["bytes"], "bytes")</code> <code>print("Time taken : %.3f s" % r["time"])</code> <code>print("Goodput : %.0f bytes/s" % r["goodput"])</code> <code>print("Packets sent :", r["packets_sent"])</code> <code>print("Retransmissions :", r["retransmissions"])</code> <code>print("Packets dropped :", r["dropped"])</code>	Print the report.
<code>print("SHA-256 match :", "YES - file is intact" if r["intact"] else "NO - file is broken!")</code>	The final proof: prints YES if the received file is identical to the original.
<code>if __name__ == "__main__":</code> <code>main()</code>	Run <code>main()</code> only when this file is executed directly (not when experiments import <code>run_transfer</code> from it).

Viva questions you may be asked (with easy answers)

Q. How does Go-Back-N differ from Stop-and-Wait?

A. Stop-and-Wait sends one packet per round trip. GBN keeps up to `window` packets in flight, so the pipe stays full and goodput is higher, especially when there is round-trip delay.

Q. What is a cumulative ACK?

A. ACK n means all packets with sequence number below n arrived. One ACK can confirm many packets, and a lost ACK is often 'repaired' by a later ACK.

Q. Why does the GBN receiver discard out-of-order packets?

A. It keeps no buffer; it only remembers 'expected'. This makes the receiver very simple but wastes packets: the sender must send them again.

Q. What happens on a timeout in GBN?

A. The sender resends every packet from base up to next_seq (the whole window). That is why it is called Go-Back-N.

Q. Why is GBN worse than Selective Repeat when loss increases?

A. One loss triggers many retransmissions of packets that were actually received fine. With more loss and larger windows the wasted resends grow, so goodput drops faster. Our plots show this.

Q. What is base and next_seq?

A. base = oldest unacknowledged packet; next_seq = next new packet to send. next_seq - base is the number of packets in flight and must not exceed window.

Q. Why one timer instead of one per packet?

A. In GBN if the oldest packet is fine everything after it is also fine to wait for; when the oldest packet's timer fires we resend everything anyway. So one timer for the oldest packet is enough.

Q. Why does the receiver send the ACK even for a discarded packet?

A. The repeated ACK (same number) tells the sender which packet is still missing. It is also how a lost ACK is recovered.

Q. Why does main.py use threads?

A. Sender and receiver are in the same program. The receiver runs in a background thread so both can work at the same time, like two separate computers.

Q. What is goodput?

A. Useful data delivered per second = file size / transfer time. Retransmitted copies do not count, so it measures real progress, unlike raw throughput.

Q. How can you swap protocols without changing main.py?

A. All three modules have the same send() and receive() function signatures, and PROTOCOLS maps names to modules.

Q. Why bind to port 0?

A. The operating system then picks a free port automatically, so runs never clash on ports.